COMPUTER PROGRAMMING ASSIGNMENT

ICS2102 / HBC2204 / HCB2351

Student Registration Number

HDB212-D1-0077/2026

Total Marks: 45

Prepared project report containing problem statements, algorithms and source code.

1. Introduction

This report presents the three C programming tasks in the supplied assignment: arrays; structures and functions; and pointers with dynamic memory allocation. Each program prints the student's registration number (HDB212-D1-0077/2026) as part of its output.

2. Section A – Arrays (15 Marks)

Problem statement

Create a C program that stores the marks of 10 students in an array. The program reads the marks using a loop, calculates the average mark, determines the highest and lowest marks, and displays all marks greater than the average.

Algorithm

1. Start the program.
2. Declare an array for ten student marks and variables for sum, average, highest, lowest and count.
3. Use a loop to read ten marks and add each mark to the sum.
4. Calculate average = sum / 10.
5. Initialize highest and lowest with the first mark; loop through the remaining marks to update them.
6. Display the average, highest and lowest marks.
7. Loop through the array and display each mark that is greater than the average.
8. Display a message if no mark is above average, then stop.

Flowchart outline

Start → Read 10 marks in a loop → Calculate sum and average → Find highest and lowest → Display marks above average → Stop

Source code

```
#include <stdio.h>

#define SIZE 10

int main(void) {     float
marks[SIZE];     float sum =
0.0f, average;     float
highest, lowest;     int
count = 0;

    printf("========================================\n");
printf("          SECTION A: ARRAYS\n");
printf("Registration Number: HDB212-D1-0077/2026\n");
printf("========================================\n");

    /* Read marks for ten students and calculate their sum. */
for (int i = 0; i < SIZE; i++) {         printf("Enter marks for
student %d: ", i + 1);         if (scanf("%f", &marks[i]) != 1) {
fprintf(stderr, "Invalid input. Please enter a number.\n");
return 1;         }         sum += marks[i];     }

    /* Calculate the average mark. */
average = sum / SIZE;

    /* Find the highest and lowest marks.
*/     highest = marks[0];     lowest =
marks[0];

    for (int i = 1; i < SIZE; i++)
{         if (marks[i] > highest)
{             highest = marks[i];
}         if (marks[i] < lowest) {
lowest = marks[i];         }
    }

    /* Display the summary and marks above the average.
*/     printf("\n========== RESULTS SUMMARY
==========\n");     printf("Average Mark: %.2f\n",
average);     printf("Highest Mark: %.2f\n", highest);
printf("Lowest Mark : %.2f\n", lowest);

    printf("\nMarks Greater Than Average (%.2f):\n",
average);     for (int i = 0; i < SIZE; i++) {         if
(marks[i] > average) {             printf("Student %d:
%.2f\n", i + 1, marks[i]);             count++;         }
}

    if (count == 0) {         printf("No marks were greater
than the average.\n");     }

    printf("\nRegistration Number: HDB212-D1-
0077/2026\n");     return 0;
}
```

3. Section B – Structures and Functions (15 Marks)

Problem statement

Design a Student Record Management Program using structures. The structure contains Admission Number, Student Name, Course, Age and CAT Marks. Functions capture student information, display student information and calculate the average CAT marks. The program stores information for five students.

Algorithm

1. Start the program and declare an array of five Student structures.
2. Each structure stores admission number, student name, course, age and CAT marks.
3. Call the input function to capture the five student records.
4. Call the display function to show each student's information.
5. Call the average function; add the five CAT marks and divide the total by five.
6. Display the average CAT mark and stop.

Flowchart outline

Start → Declare 5 Student records → Capture records using input function → Display records using display function → Calculate average CAT marks using average function → Stop

Source code

```
#include <stdio.h>

#define MAX_STUDENTS 5

/* Structure containing the fields required by the assignment.
*/ struct Student {     char admissionNumber[30];     char
name[50];     char course[60];     int age;     float catMarks;
};

/* Function prototypes. */ int inputStudents(struct Student
students[], int count); void displayStudents(const struct Student
students[], int count); float calculateAverage(const struct
Student students[], int count);

int main(void) {     struct Student
students[MAX_STUDENTS];

    printf("========================================\n");
printf(" SECTION B: STUDENT RECORD MANAGEMENT\n");
printf("Registration Number: HDB212-D1-0077/2026\n");
printf("========================================\n");

    if (!inputStudents(students, MAX_STUDENTS))
{         return 1;     }
displayStudents(students, MAX_STUDENTS);

    printf("\nAverage CAT Marks: %.2f\n",
calculateAverage(students, MAX_STUDENTS));
printf("Registration Number: HDB212-D1-0077/2026\n");

    return 0;
}

/* Capture the information for five students. */ int
inputStudents(struct Student students[], int count) {
for (int i = 0; i < count; i++) {         printf("\nEnter
details for student %d:\n", i + 1);

        printf("Admission Number: ");         if (scanf("
%29[^\n]", students[i].admissionNumber) != 1) {
fprintf(stderr, "Invalid admission number.\n");
return 0;         }

        printf("Student Name: ");         if (scanf("
%49[^\n]", students[i].name) != 1) {
fprintf(stderr, "Invalid student name.\n");
return 0;         }

        printf("Course: ");         if (scanf("
%59[^\n]", students[i].course) != 1) {
```

```
fprintf(stderr, "Invalid course.\n");             return
0;         }

        printf("Age: ");         if
(scanf("%d", &students[i].age) != 1) {
fprintf(stderr, "Invalid age.\n");
return 0;         }

        printf("CAT Marks: ");         if
(scanf("%f", &students[i].catMarks) != 1) {
fprintf(stderr, "Invalid CAT mark.\n");
return 0;         }     }     return 1; }

/* Display all five student records. */ void displayStudents(const
struct Student students[], int count) {     printf("\n==========
STUDENT RECORDS ==========\n");     for (int i = 0; i < count; i++)
{         printf("\nStudent %d\n", i + 1);
printf("Admission Number: %s\n", students[i].admissionNumber);
printf("Student Name    : %s\n", students[i].name);
printf("Course          : %s\n", students[i].course);
printf("Age             : %d\n", students[i].age);
printf("CAT Marks       : %.2f\n", students[i].catMarks);     }
}

/* Calculate and return the average CAT mark. */ float
calculateAverage(const struct Student students[], int count) {
float total = 0.0f;

    for (int i = 0; i < count; i++) {
total += students[i].catMarks;     }

    return total / count;
}
```

4. Section C – Pointers and Dynamic Memory Allocation(15 Marks)

Problem statement

Write a C program that dynamically allocates memory for 15 integers using malloc(), reads values into the allocated memory, calculates the total and average, and frees the allocated memory before program termination.

Algorithm

1. Start the program and declare a pointer to integer and a total variable.
2. Use malloc() to allocate memory for 15 integers.
3. Check whether malloc() returned NULL; if allocation failed, display an error and stop.
4. Use a loop to read 15 integers through the pointer and add them to the total.
5. Calculate average = total / 15.
6. Display the total and average.
7. Call free() to release the allocated memory, then stop.

Flowchart outline

Start → malloc() for 15 integers → Check allocation → Read 15 integers and total them → Calculate average → Display results → free() memory → Stop

Source code

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 15

int main(void) {
int *numbers;     long
long total = 0;
double average;

    printf("========================================\n");
printf(" SECTION C: POINTERS AND DYNAMIC MEMORY\n");
printf("Registration Number: HDB212-D1-0077/2026\n");
printf("========================================\n");

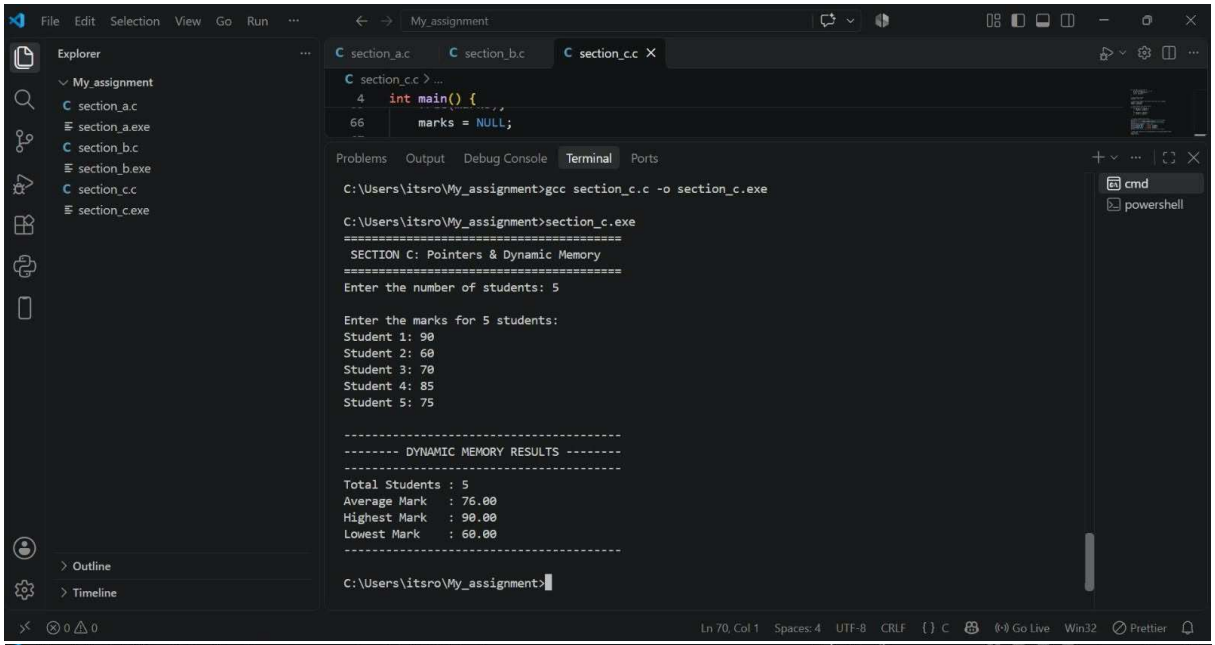
    /* Allocate memory for exactly 15 integers. */
numbers = malloc(SIZE * sizeof *numbers);     if
(numbers == NULL) {         fprintf(stderr, "Memory
allocation failed.\n");         return 1;     }

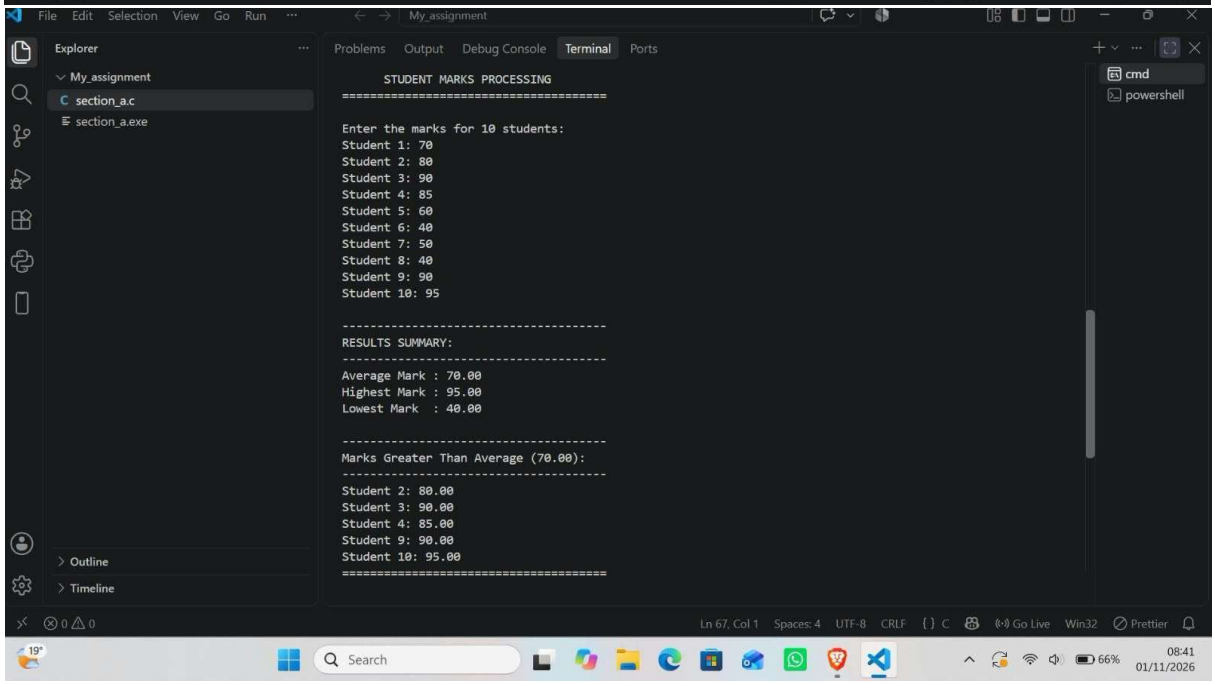
    /* Read 15 integers and calculate their total. */
printf("\nEnter %d integer values:\n", SIZE);     for (int i = 0; i <
SIZE; i++) {         printf("Enter integer %d: ", i + 1);         if
(scanf("%d", numbers + i) != 1) {             fprintf(stderr, "Invalid
input. Please enter an integer.\n");             free(numbers);
return 1;         }         total += *(numbers + i);     }

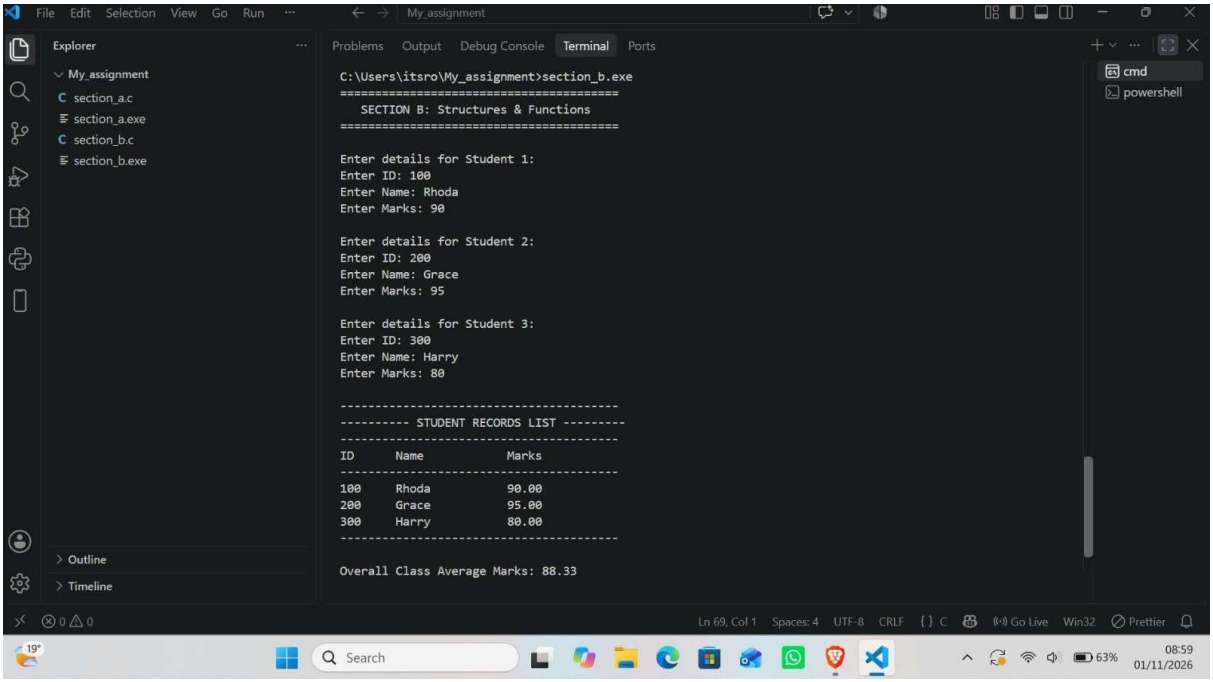
    /* Calculate and display the average. */
average = (double) total / SIZE;

    printf("\n========== RESULTS ==========\n");
printf("Total of 15 integers: %lld\n", total);
printf("Average: %.2f\n", average);
printf("Registration Number: HDB212-D1-0077/2026\n");

    /* Release dynamically allocated memory. */
free(numbers);     numbers = NULL;
printf("Allocated memory has been freed.\n");
return 0; }
```


File Edit Selection View Go Run
My_assignment
Explorer
My_assignment
section_a.c
section_a.exe
section_b.c
section_b.exe
section_c.c
section_c.exe
Outline
Timeline
section_a.c
section_b.c
section_c.c
section_c.c > ...
4 int main() {
66 marks = NULL;
Problems Output Debug Console Terminal Ports
cmd
powershell
C:\Users\itsro\My_assignment>gcc section_c.c -o section_c.exe
C:\Users\itsro\My_assignment>section_c.exe
SECTION C: Pointers & Dynamic Memory
Enter the number of students: 5
Enter the marks for 5 students:
Student 1: 90
Student 2: 60
Student 3: 70
Student 4: 85
Student 5: 75
-------- DYNAMIC MEMORY RESULTS --------
Total Students : 5
Average Mark : 76.00
Highest Mark : 90.00
Lowest Mark : 60.00
C:\Users\itsro\My_assignment>
Ln 70, Col 1 Spaces: 4 UTF-8 CRLF { } C Go Live Win32 Prettier


File Edit Selection View Go Run
My_assignment
Explorer
My_assignment
section_a.c
section_a.exe
Outline
Timeline
Problems Output Debug Console Terminal Ports
cmd
powershell
STUDENT MARKS PROCESSING
Enter the marks for 10 students:
Student 1: 70
Student 2: 80
Student 3: 90
Student 4: 85
Student 5: 60
Student 6: 40
Student 7: 50
Student 8: 40
Student 9: 90
Student 10: 95
RESULTS SUMMARY:
Average Mark : 70.00
Highest Mark : 95.00
Lowest Mark : 40.00
Marks Greater Than Average (70.00):
Student 2: 80.00
Student 3: 90.00
Student 4: 85.00
Student 9: 90.00
Student 10: 95.00
Ln 67, Col 1 Spaces: 4 UTF-8 CRLF { } C Go Live Win32 Prettier
Search
66%
08:41
01/11/2026


File Edit Selection View Go Run
My_assignment
Explorer
My_assignment
section_a.c
section_a.exe
section_b.c
section_b.exe
Problems Output Debug Console Terminal Ports
cmd
powershell
C:\Users\itsro\My_assignment>section_b.exe
SECTION B: Structures & Functions
Enter details for Student 1:
Enter ID: 100
Enter Name: Rhoda
Enter Marks: 90
Enter details for Student 2:
Enter ID: 200
Enter Name: Grace
Enter Marks: 95
Enter details for Student 3:
Enter ID: 300
Enter Name: Harry
Enter Marks: 80
---------- STUDENT RECORDS LIST ---------
ID Name Marks
100 Rhoda 90.00
200 Grace 95.00
300 Harry 80.00
Overall Class Average Marks: 88.33
Outline
Timeline
Ln 69, Col 1 Spaces: 4 UTF-8 CRLF C Go Live Win32 Prettier
Search
63%
08:59
01/11/2026